
7) WHAT IS QUALITY GATE

Quality Gate is a set of rules that determines whether the code is good enough to deploy.

Example Quality Gate

Condition

Coverage < 80% → FAIL

Bugs > 0 → FAIL

Vulnerabilities > 0 → FAIL

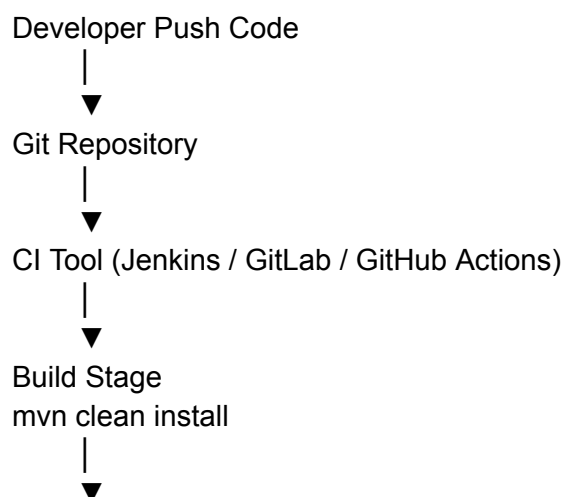
Duplications > 5% → FAIL

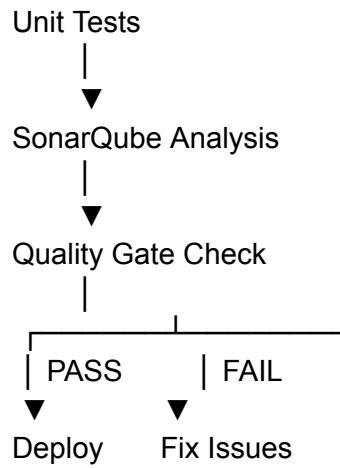
If conditions fail

Build fails.

8) SONARQUBE IN CI/CD PIPELINE

Typical CI/CD pipeline





9) IN WHICH PHASE SONARQUBE RUNS

Typically executed in

Build / Code Quality phase

Example Jenkins pipeline

Stage 1

Compile

Stage 2

Unit Tests

Stage 3

SonarQube Analysis

Stage 4

Quality Gate

Stage 5

Deploy

Example command

```
mvn clean verify sonar:sonar
```

10) COMMON ISSUES DETECTED IN REAL PROJECTS

1) Hardcoded password

```
String password = "admin123";
```

Security vulnerability

2) System.out.println

```
System.out.println("User created");
```

Bad practice

Fix

Use Logger

3) Empty catch block

```
try {  
    service.call();  
}  
catch(Exception e) {  
}
```

Issue

Exception ignored

4) Duplicated code

Same method written multiple times.

5) Unused variables

```
int count = 0;
```

Never used.

11) SONARQUBE RULES

Rules define how code is analyzed.

Example rule categories

Reliability

Security

Maintainability

Example Java rule

S106

Avoid using System.out

12) SONARQUBE QUALITY PROFILES

Quality Profile = Set of rules applied to project.

Example

Java Quality Profile

contains 1000+ rules.

Companies often create custom profiles.

13) TECHNICAL DEBT

Technical debt = Time required to fix issues.

Example

Issue

Large method

Estimated fix time

30 minutes

Total project technical debt

2 days

14) SONARQUBE PULL REQUEST ANALYSIS

When developer creates PR

SonarQube analyzes only changed code.

Example

GitHub PR

Shows

New bugs

New vulnerabilities

Coverage drop

Helps prevent bad code merging.

15) BEST PRACTICES IN REAL PROJECTS

1) Fix issues before merge

2) Maintain coverage > 80%

3) Avoid code duplication

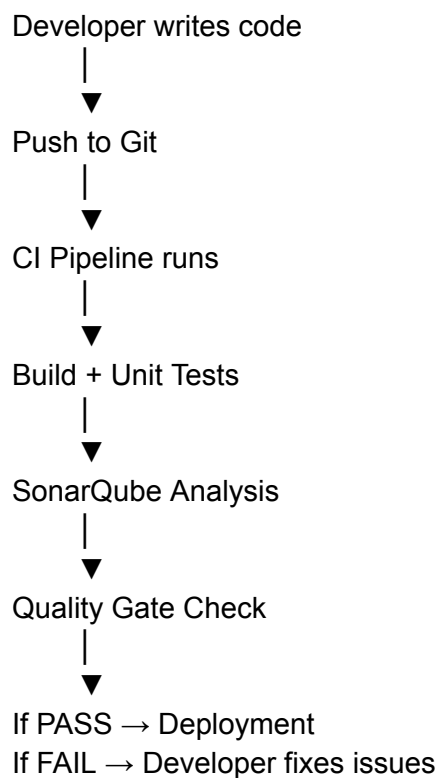
4) Follow coding standards

5) Enforce Quality Gates

16) INTERVIEW ONE-LINE ANSWER

SonarQube is a static code analysis platform that analyzes source code during the build process to detect bugs, security vulnerabilities, code smells, duplication, and enforce quality gates to maintain high code quality.

17) REAL PROJECT FLOW SUMMARY



unit and Integration testing

Scenario based

1) Self Invocation Problem in Spring @Transactional

Problem

When a method annotated with @Transactional is called from another method within the same class, the transaction may not be applied.

Example

@Service

```
public class PaymentService {
```

```
    @Transactional
```

```
    public void methodA() {
```

```
        // DB operation
```

```
    }
```

```
    public void methodB() {
```

```
        methodA(); // internal call
```

```
    }
```

```
}
```

If methodB() is called, the transaction on methodA() will NOT start.

Reason

Spring implements @Transactional using proxy-based AOP.

When a method is called from outside the bean:

Client → Spring Proxy → Target Method

The proxy intercepts the call and starts the transaction.

But when a method inside the same class calls another method:

methodB() → this.methodA()

The call bypasses the proxy and directly invokes the method.

Since the proxy is skipped, Spring cannot apply transactional logic.

Solution

1. Move the transactional method to another service class.

@Service

```
class TransactionService {
```

```
@Transactional
public void methodA() {}
}
```

2. Call the method through another bean (proxy).

```
@Autowired
TransactionService txService;

txService.methodA();
```

Key Point

Spring creates a proxy for the bean (class).

Transactional behavior is applied only when the method call goes through the proxy.

Internal method calls bypass the proxy, so `@Transactional` is not applied.

2) Default Method Conflict in Multiple Interfaces (Diamond Problem)

Issue

When a class implements two interfaces that both contain the same default method with the same signature, Java cannot decide which implementation should be used.

Reason

Both interfaces provide their own default implementation of the same method, which creates ambiguity for the implementing class.

Example

```
interface A {
    default void m1() {
        System.out.println("A m1()");
    }
}
```

```
interface B {
    default void m1() {
        System.out.println("B m1()");
    }
}
```

```
class Test implements A, B {}
```

Compilation Error

class Test inherits unrelated defaults for m1() from types A and B

Solution

The implementing class must override the method and explicitly choose which interface method to use.

class Test implements A, B {

```
    @Override
    public void m1() {
        A.super.m1(); // or B.super.m1()
    }
}
```

Key Point

If multiple interfaces provide the same default method, the implementing class must override it to resolve the conflict.

3) LazyInitializationException (Hibernate / JPA)

Problem

LazyInitializationException occurs when Hibernate tries to load a lazily fetched entity after the Hibernate session has already been closed.

Error Example

org.hibernate.LazyInitializationException:
could not initialize proxy - no session

Reason

In JPA, relationships are often configured with LAZY loading.

Example:

```
@Entity
class Order {

    @Id
    Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    Customer customer;
}
```

When Order is fetched, Hibernate loads only the Order data:

```
SELECT * FROM orders WHERE id = ?
```

The 'customer' field is not loaded immediately. Instead Hibernate creates a proxy object for Customer.

Later when the code accesses:

```
order.getCustomer().getName()
```

Hibernate tries to run another query:

```
SELECT * FROM customer WHERE id = ?
```

If the Hibernate session is already closed at this time, Hibernate cannot execute the query and throws `LazyInitializationException`.

Solutions

1. JOIN FETCH

Load related entities in the same query.

```
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.id = :id")
```

2. DTO Projection

Fetch only required fields and return a DTO instead of entity.

3. @Transactional

Keep the Hibernate session open so lazy properties can be loaded within the transaction.

Best Practice

In production systems, DTO projection or JOIN FETCH is preferred to avoid lazy loading issues and unnecessary database queries.

4) Memory Leak in Spring Boot

Scenario

Application memory usage keeps increasing over time and does not decrease even after requests are completed. Eventually the application may crash with `OutOfMemoryError`.

Example Error

java.lang.OutOfMemoryError: Java heap space

A memory leak happens when objects are no longer needed but are still being referenced, so the JVM Garbage Collector cannot free them.

1. Caching Without Eviction

Problem

If caching is used but old entries are never removed, the cache keeps growing and consumes more memory.

Example:

```
@Cacheable("products")
public Product getProduct(Long id) {
    return productRepository.findById(id).get();
}
```

If thousands of products are requested, the cache keeps storing new objects without removing old ones.

Result

Memory usage keeps increasing.

Solution

Use cache eviction or TTL (time to live).

Example:

```
@CacheEvict(value = "products", key = "#id")
public void updateProduct(Product product) {
    productRepository.save(product);
}
```

Or configure cache expiration in Redis/EhCache.

2. Static Collections

Problem

Static variables live for the entire lifecycle of the application.

Example:

```
public class UserStore {  
  
    static List<User> users = new ArrayList<>();  
  
}
```

If the application keeps adding users:

```
users.add(newUser);
```

The list keeps growing and objects are never removed.

Result

Garbage Collector cannot remove these objects because static references still exist.

Solution

Avoid static collections for storing runtime data.
Use proper cache frameworks with eviction policies.

3. Thread Pools Not Closed

Problem

If thread pools are created but not shut down, threads remain active and hold memory references.

Example:

```
ExecutorService executor = Executors.newFixedThreadPool(10);  
  
executor.submit(() -> processTask());
```

If the executor is never closed:

```
executor.shutdown();
```

Threads remain alive and memory usage increases.

Solution

Always properly shut down thread pools.

Example:

```
executor.shutdown();
```

Or use Spring-managed thread pools such as:

```
@Async
TaskExecutor
ThreadPoolTaskExecutor
```

4. Large Objects Stored in Session

Problem

Storing large objects in HTTP session keeps them in memory for a long time.

Example:

```
session.setAttribute("reportData", hugeList);
```

If many users store large data, memory usage grows.

Solution

Store minimal data in session or move data to database/cache.

5. Unclosed Resources

Problem

Not closing resources like streams, files, or DB connections can also increase memory usage.

Example:

```
InputStream input = new FileInputStream(file);
```

If not closed, resources remain allocated.

Solution

Use try-with-resources.

Example:

```
try(InputStream input = new FileInputStream(file)) {  
    // process  
}
```

Interview Summary

Memory leak in Spring Boot occurs when objects remain referenced and cannot be garbage collected. Common causes include unbounded caching, static collections, unclosed thread pools, large session objects, and unclosed resources. Proper resource management and cache eviction strategies help prevent it.

5) N+1 Query Problem (Hibernate / JPA)

Definition

The N+1 query problem occurs when an application executes 1 query to fetch parent records and then executes additional queries (one per record) to fetch related entities.

This results in excessive database calls and poor performance.

Example Scenario

Entities:

```
@Entity  
class Customer {  
  
    @Id  
    Long id;  
  
    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)  
    List<Order> orders;  
}
```

```
@Entity  
class Order {
```

```
@Id
Long id;

@ManyToOne
Customer customer;
}
```

Code Example

Repository:

```
List<Customer> customers = customerRepository.findAll();
```

Then accessing orders:

```
for(Customer c : customers){
    System.out.println(c.getOrders().size()); //when we did this automatically extra query generated to
                                                Get related lazy data
}
```

What Happens Internally

Step 1: Hibernate loads all customers

SQL Query 1

```
SELECT * FROM customer;
```

Suppose there are 5 customers.

Step 2: When orders are accessed for each customer

Hibernate executes additional queries:

```
SELECT * FROM orders WHERE customer_id = 1
SELECT * FROM orders WHERE customer_id = 2
SELECT * FROM orders WHERE customer_id = 3
SELECT * FROM orders WHERE customer_id = 4
SELECT * FROM orders WHERE customer_id = 5
```

Total queries executed:

1 + 5 = 6 queries

This is called the N+1 query problem.

Why It Happens

Because of lazy loading. Hibernate loads related entities only when they are accessed, so each access triggers a new database query.

Performance Impact

If there are 1000 customers:

1 query for customers
1000 queries for orders

Total queries = 1001

This causes:

- high database load
 - slow API responses
 - poor scalability
-

Solution 1: JOIN FETCH

Load parent and child entities in a single query. And call this method from service

```
@Query("SELECT c FROM Customer c JOIN FETCH c.orders")  
List<Customer> findCustomersWithOrders();
```

SQL generated:

```
SELECT c.*, o.*  
FROM customer c  
JOIN orders o ON c.id = o.customer_id
```

Solution 2: DTO Projection

Fetch only required fields.

```
@Query("""
```

```
SELECT new com.example.CustomerDTO(c.id, c.name, o.id)
FROM Customer c
JOIN c.orders o
""")
```

This avoids entity loading and improves performance.

Solution 3: EntityGraph

```
@EntityGraph(attributePaths = {"orders"})
List<Customer> findAll();
```

This tells Hibernate to fetch the orders along with customers.

Interview Summary

N+1 query problem occurs when one query fetches parent records and additional queries are executed for each related entity due to lazy loading.

Solutions include JOIN FETCH, DTO projections, and EntityGraph to load related data efficiently in fewer queries.

6) Using Retry and Circuit Breaker Together in Microservices

Scenario

In a microservice architecture, Service A may call another service (Service B). If Service B is temporarily unavailable or slow, the request may fail.

To make the system more resilient, we can apply both Retry and Circuit Breaker from Resilience4j on the same method.

Retry

Retry attempts the request again if the first attempt fails. This helps handle temporary failures such as network issues or short service downtime.

Circuit Breaker

Circuit breaker monitors failures. If failures exceed a threshold, the circuit opens and stops further calls to the failing service. Instead, a fallback response is returned.

Using both together ensures:

- Retry handles temporary failures.
- Circuit breaker protects the system from repeated failures.

Execution Flow

Client



Service A



Retry attempts (multiple calls if failure)



Circuit breaker monitors failure rate



If failure threshold exceeded → circuit opens



Fallback method executed

Example Code

@Service

```
public class PaymentService {
```

```
    @Retry(name = "paymentService")
```

```
    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
```

```
    public Payment getPayment(Long id) {
```

```
        return paymentClient.getPayment(id);
```

```
    }
```

```
    public Payment fallbackPayment(Long id, Exception ex) {
```

```
        return new Payment("Payment service temporarily unavailable");
```

```
    }
```

```
}
```

In this example:

- Retry will attempt the request multiple times.
- If failures continue, the circuit breaker may open.
- If the circuit is open, the fallback method will execute.

Configuration (application.yml)

```
resilience4j:
```

```
  retry:
```

```
    instances:
```

```
paymentService:  
  maxAttempts: 3  
  waitDuration: 1s
```

```
circuitbreaker:  
  instances:  
    paymentService:  
      failureRateThreshold: 50  
      slidingWindowSize: 10  
      waitDurationInOpenState: 10s
```

Configuration Meaning

Retry Configuration

- maxAttempts: Maximum number of retry attempts.
- waitDuration: Time delay between retry attempts.

Circuit Breaker Configuration

- failureRateThreshold: Percentage of failures that will open the circuit.
- slidingWindowSize: Number of calls used to calculate failure rate.
- waitDurationInOpenState: Time the circuit remains open before trying again.

Benefits

- Improves fault tolerance in microservices.
- Prevents cascading failures.
- Provides fallback responses when services are unavailable.

Best Practice

In production microservices systems, resilience patterns often combine:

Timeout + Retry + Circuit Breaker + Fallback

to ensure stable and reliable service communication.

7) REST API Design Issue

Problem:

API returns HTTP 200 OK even when an error occurs. This is incorrect REST design because HTTP status codes should represent the result of the request.

Reason:

Spring Boot returns 200 by default if the controller method completes successfully and no exception is thrown. Some developers also manually return `ResponseEntity.ok()` even in error cases.

Solution:

Use proper HTTP status codes.

400 Bad Request → Invalid client input

404 Not Found → Resource not found

500 Internal Server Error → Server failure

Use `@ControllerAdvice` to implement global exception handling and return consistent error responses across the application.

8) Production Debugging Scenario – Random 500 Errors

Problem:

API randomly returns HTTP 500 Internal Server Error in production.

Possible causes:

Database issues, external service failures, infrastructure problems, or code exceptions.

Debugging steps:

1. Check application logs to identify stack trace and error.
2. Trace request using correlation ID across microservices.
3. Monitor database queries and connection pool usage.
4. Verify external services and APIs.
5. Check system metrics like CPU, memory, and container health.
6. If issue started after deployment, rollback to previous stable release.

=====

SPRING BOOT INTERVIEW PREPARATION NOTES

(3–5 Years Java / Spring Boot Developer)

=====

=====

SECTION 1 — 40 SPRING BOOT QUESTIONS (PRODUCT COMPANIES)

=====

CORE SPRING BOOT

1. What happens internally when a Spring Boot application starts?

Flow:

```
SpringApplication.run()
|
Create Application Context
|
Component Scan
|
Bean Creation
|
AutoConfiguration
|
Embedded Server Start (Tomcat)
```

2. Difference: Spring vs Spring Boot

Spring

- Requires manual configuration
- XML or Java config
- External server setup

Spring Boot

- Auto configuration
- Embedded server
- Starter dependencies

3. What is AutoConfiguration?

Spring Boot automatically configures beans based on:

- classpath dependencies
- properties configuration

Example:

If spring-web dependency exists

→ DispatcherServlet auto configured.

4. @SpringBootApplication internally

@SpringBootApplication =

```
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

5. Difference between stereotypes

@Component
Generic bean

@Service
Business layer bean

@Repository
DAO layer bean + exception translation

@Controller
MVC controller

6. What is Spring Boot Starter?

Predefined dependency bundle.

Example:

spring-boot-starter-web

Includes
- spring-web
- jackson
- tomcat

7. application.properties vs application.yml

properties

```
server.port=8080
spring.datasource.url=jdbc:mysql://
```

yaml

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://
```

8. Component Scanning

Spring scans packages from the main class package.

@SpringBootApplication
|
scan base package
|
create beans for annotated classes

BEAN LIFECYCLE

9. Spring Bean Lifecycle

Instantiate bean
|
Populate dependencies
|
@PostConstruct
|
Bean ready
|
@PreDestroy

10. Singleton vs Prototype

Singleton
- one instance per container

Prototype
- new instance each request

11. @PostConstruct

Runs after bean initialization.

```
@PostConstruct  
void init() {  
    // initialization logic  
}
```

12. Dependency Injection Types

Constructor Injection
Setter Injection